

DEVELOPPEMENT D'UNE APPLICATION DESKTOP : IMPLEMENTATION DES ALGORITHMES DE RECHERCHE OPERATIONNELLE EN RESEAUX ET TELECOMMUNICATIONS

Réalisé par

ZAIDOURY Najlae

Période : Mai 2025

Encadré par:

Mme LMKHALET Mouna

Année universitaire:

2024-2024

BIENVENUE,

Je souhaite exprimer ma sincère et profonde gratitude à Madame LMKHALET Mouna pour la qualité de son encadrement, la pertinence de ses conseils ainsi que sa disponibilité constante tout au long de ce projet. Son accompagnement attentif et bienveillant a grandement contribué à l'enrichissement de ce travail et à ma progression personnelle et académique.

Je tiens également à remercier chaleureusement mes camarades pour la richesse de nos échanges, leur esprit de collaboration et leur soutien. Enfin, je suis profondément reconnaissante envers mes proches, dont le soutien moral inébranlable a été un pilier essentiel tout au long de cette période exigeante.

02. RESUME:

Ce projet s'inscrit dans le cadre du module de Recherche Opérationnelle à l'École Mohammadia d'Ingénieurs. Il a pour objectif de concevoir et de développer une application interactive permettant l'implémentation des principaux algorithmes de recherche opérationnelle, notamment Dijkstra, Kruskal, Welsh powel..., en mettant particulièrement en lumière leur applicabilité dans le secteur des télécommunications.

La méthodologie adoptée repose sur un cycle de développement itératif. Après une phase d'analyse approfondie des besoins fonctionnels et techniques, plusieurs diagrammes ont été élaborés afin de modéliser les différentes composantes du système. L'interface utilisateur a été conçue en Python, à l'aide de Tkinter, avec une attention particulière portée à l'ergonomie et à l'accessibilité de l'outil.



Les résultats obtenus démontrent que l'application facilite non seulement l'assimilation théorique des algorithmes étudiés, mais permet également leur expérimentation dans un cadre interactif. Cette approche met en évidence le fait que ces algorithmes ne se limitent pas à une abstraction mathématique ou académique, mais constituent des leviers puissants d'optimisation applicables à divers domaines concrets:

de la gestion des ressources à la planification stratégique , améliorant ainsi l'efficacité, la performance et l'expérience utilisateur dans des environnements complexes tels que les télécommunications.

02. RESUME AN:

This project was carried out as part of the [module name] course at the Mohammadia School of Engineering. Its main objective was to design and develop an interactive application implementing key operations research algorithms, such as [name 3 or 4 key algorithms], with a particular focus on their relevance to the field of telecommunications.

The methodology adopted followed an iterative development cycle. After an in-depth analysis of the functional and technical requirements, several UML diagrams were created to model the system's architecture. The user interface was developed in Python, using [name of libraries], with special attention given to usability and user experience.



The results demonstrate that the developed tool not only facilitates the theoretical understanding of the studied algorithms but also enables hands-on experimentation in an interactive environment. This approach highlights the fact that these algorithms are far more than abstract mathematical constructs; they are powerful tools that can be applied across a wide range of real-world domains

from resource management to strategic planning — thereby enhancing efficiency, performance, and overall user experience in complex environments such as telecommunications.

SOMMAIRE

01

INTRODUCTION

06

- 1.1. Contexte du projet
- 1.2. Problématique
- 1.3. Objectifs du projet
- 1.4. Méthodologie adoptée

02

ANALYSE DU BESOIN ET CADRAGE DU PROJET

08

- 2.1. Origine et motivation du projet
- 2.2. Besoins identifiés
- 2.3. Objectifs fonctionnels et techniques
- 2.4. Contraintes et périmètre du projet

03

CONCEPTION REALISATION DE LA SOLUTION

10

- 3.1. Choix des algorithmes
- 3.2. Scénarios d'utilisation
- 3.3. Architecture logicielle simple
- 3.4. Justification des choix de conception

04

RESULTATS

14

Bonus

05

RÉSULTATS, CONCLUSION ET PERSPECTIVES

21

- 5.1. Résultats obtenus et comparaison avec les objectifs
- 5.2. Apports personnels et compétences acquises
- 5.3. Limites du projet
- 5.4. Améliorations possibles et perspectives futures

06

BIBLIOGRAPHIE

22

1. Introduction:

1.1 Contexte du projet:

La recherche opérationnelle est un domaine essentiel dans l'optimisation des systèmes complexes, qu'ils soient industriels, logistiques ou technologiques. Avec l'évolution constante des outils numériques, l'implémentation de ces techniques à travers des langages de programmation devient de plus en plus courante, notamment en Python grâce à sa richesse en bibliothèques scientifiques.

Ce projet s'inscrit dans cette dynamique, en mettant en œuvre des algorithmes de recherche opérationnelle dans une application interactive développée en Python. L'objectif est d'apporter une solution informatique simple et visuelle à des problèmes d'optimisation classiques, tout en renforçant la compréhension des concepts étudiés.

1.2 Problématique:

Les problèmes d'optimisation interviennent dans de nombreuses situations concrètes en télécommunications, comme l'allocation de la bande passante, le routage, la gestion des fréquences ou la planification des ressources. Bien qu'ils soient modélisés mathématiquement, ils restent souvent abstraits pour ceux qui ne manipulent pas directement les outils numériques.

La problématique à laquelle ce projet répond est : **Comment exploiter le langage Python pour concevoir une interface simple et intuitive permettant de résoudre efficacement des problèmes d'optimisation issus de la recherche opérationnelle ?**

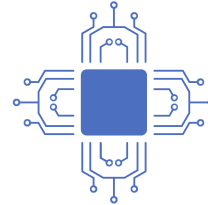


1. Introduction:

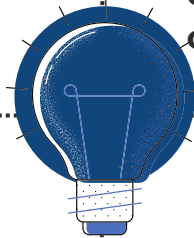
1.3 Objectifs du projet:



Concevoir une interface graphique simple pour visualiser des algorithmes d'optimisation.



Implémenter en Python un ou plusieurs algorithmes classiques de recherche opérationnelle.



Permettre à l'utilisateur d'interagir avec le système en entrant des données et en observant les résultats.



Fournir une base extensible pour intégrer d'autres méthodes ou algorithmes ultérieurement

1.4 Méthodologie adoptée:

- Étude des algorithmes de recherche opérationnelle à implémenter (algorithme du Kruskal, méthode du nord-ouest, etc.).
- Sélection des bibliothèques Python pertinentes (Tkinter,, Matplotlib, etc.).
- Conception de l'interface utilisateur pour une interaction fluide et intuitive.
- Programmation des fonctionnalités par modules, avec des tests à chaque étape, Validation à travers des exemples concrets.

02. ANALYSE DU BESOIN ET CADRAGE DU PROJET

2.1 Présentation du domaine d'application : Réseaux et Télécommunications

Le domaine des réseaux et télécommunications repose sur la gestion et l'optimisation des flux d'information à travers des infrastructures matérielles et logicielles complexes. Dans ce secteur, les problèmes d'optimisation sont omniprésents : acheminement efficace des paquets de données, allocation de fréquences, optimisation du routage, gestion des files d'attente dans les commutateurs, etc.

Ces problèmes relèvent directement de la recherche opérationnelle, qui fournit des outils mathématiques et algorithmiques pour améliorer les performances des systèmes de communication.



2.2 Identification du besoin

Les étudiants en réseaux et télécommunications sont souvent confrontés à des notions d'optimisation (ex. : routage optimal, files d'attente, bande passante) sans pouvoir visualiser concrètement les algorithmes mis en œuvre. D'autre part, les professionnels ont besoin d'outils simples pour tester rapidement des scénarios d'optimisation, sans recourir à des logiciels complexes ou payants.

Le besoin identifié est donc double :

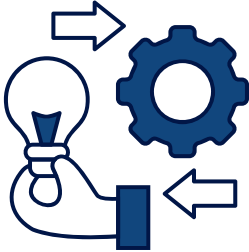
- Pédagogique : proposer un outil visuel pour faciliter la compréhension des algorithmes de recherche opérationnelle.
- Technique : fournir une base logicielle simple et légère, développée en Python, pour expérimenter des solutions à des problèmes typiques en télécoms (affectation, routage, gestion de ressources).



2.3 Cahier des charges fonctionnel:

Le projet doit répondre aux besoins suivants :

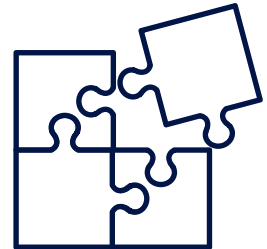
Fonctionnalités principales :



- Interface graphique permettant à l'utilisateur d'introduire ses propres données (par exemple, un tableau de coûts ou un graphe de réseau).
- Implémentation d'un ou plusieurs algorithmes d'optimisation (par exemple, algorithme de Dijkstra pour le routage, méthode de transport, etc.).
- Visualisation claire des résultats : chemin optimal, coût total, ou tableau final selon le type de problème.

Contraintes techniques:

- Le projet doit être développé uniquement en Python.
- Aucune base de données ou modélisation formelle (MCD ou UML) ne sera utilisée.
- Le projet doit rester léger, simple à exécuter sur tout ordinateur avec Python installé.



Public cible :

- Étudiants en réseaux et télécommunications.
- Enseignants souhaitant illustrer leurs cours de recherche opérationnelle.
- Toute personne curieuse de mieux comprendre les mécanismes d'optimisation.

2.4 Délimitation du projet:

Ce projet, bien qu'axé sur un domaine riche et complexe, ne prétend pas couvrir l'ensemble des problématiques des télécoms. Il se limite volontairement à :

- Une implémentation simple (non industrielle) d'algorithmes.
- Une interface utilisateur minimale (via Tkinter).
- Un nombre restreint de scénarios (par exemple, routage sur un graphe simple, résolution de problème de transport, etc.).

Cela permet de se concentrer sur l'essentiel : **comprendre, tester et visualiser les algorithmes d'optimisation, en lien avec des problématiques télécoms.**

03. CONCEPTION REALISATION DE LA SOLUTION

3.1 Langage et bibliothèques utilisées

Langage de programmation : Python

Le langage Python a été choisi pour sa simplicité, sa lisibilité et sa large gamme de bibliothèques spécialisées. Il est particulièrement adapté pour :

- L'implémentation rapide d'algorithmes.
- La manipulation de structures de données.
- La visualisation de résultats.



```
statuses = {}
for data in resp_iter:
    status = Status(
        status_id=data.id, name=data.name
    )
    statuses[status.name] = status
return statuses
```

	Bibliothèques utilisées
tkinter	pour la création d'une interface utilisateur simple (aussi biblio PIL pour coloration dégradée)
matplotlib	pour visualiser des graphes ou des matrices et tableaux
networkx	pour la gestion de graphes, utile dans des algorithmes comme Dijkstra.

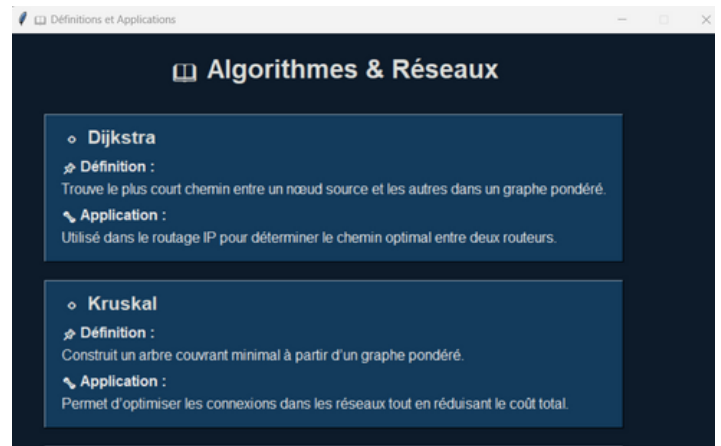
3.2 Architecture générale de la solution

Le code était structurée de la façon suivante:

- 1) def generate_subtle_gradient(): #pour une interface plus esthétique
- 2) def open_algo_description(): #en appuyant sur le bouton ENTRER on va avoir une fiche explicative de chaque algo et son intérêt dans le domaine de reseaux
- 3) def lancer_dijkstra(): #fait appel a l'algorithme dijkstra
- 4) on a fait appel a tous les algo comme dijkstra (kruskal, welsh powel, bellman ford, potentiel metra, ford fulkerson, moindre cout, nords ouest et stepping stone
- 5) def main_interface() l'interface de notre application avec chaque bouton lié a son algorithme

03. CONCEPTION REALISATION DE LA SOLUTION

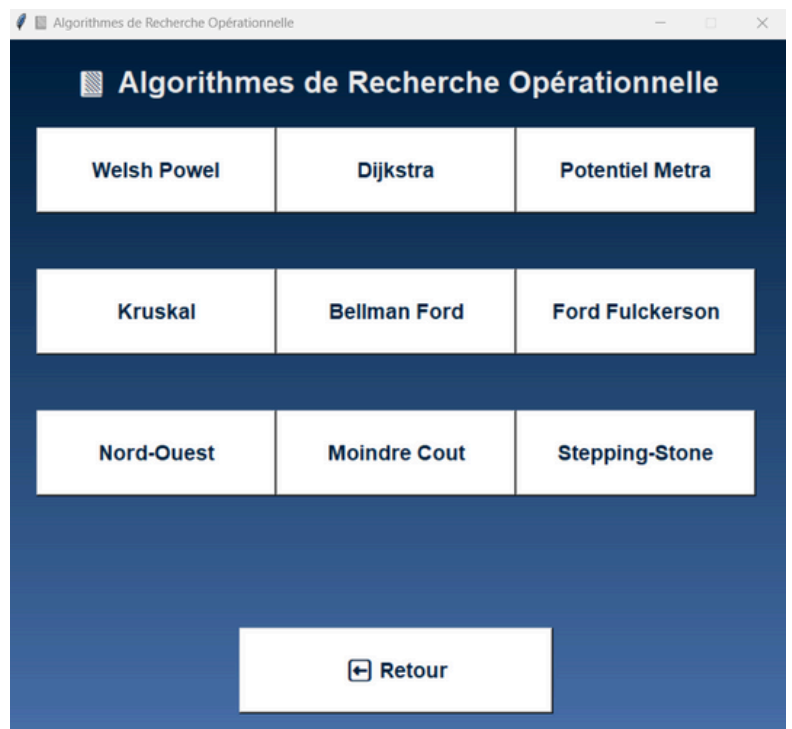
3.3 Description de l'interface utilisateur



L'interface est conçue pour être intuitive et fonctionnelle, même pour des utilisateurs n'ayant pas de connaissances en programmation.

Elle comprend :

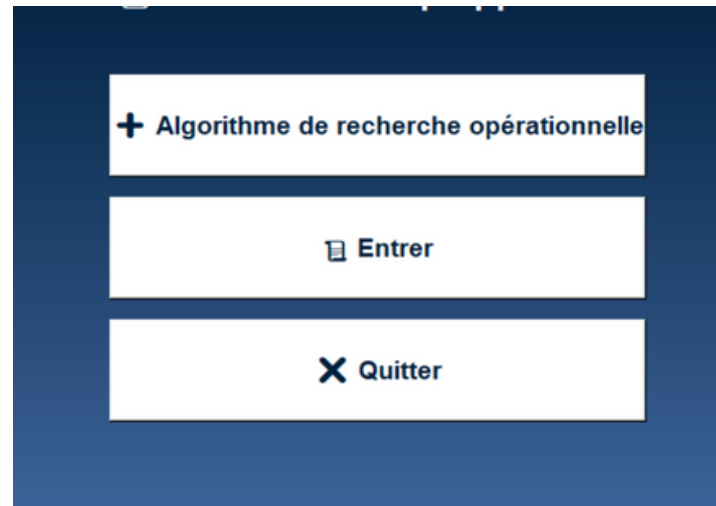
- Un champ de saisie ou un bouton d'importation de données (matrice, graphe...).
- Des boutons pour lancer l'exécution des algorithmes.
- Une zone d'affichage des résultats (texte, tableau ou visualisation graphique).
- Des messages d'erreur ou d'alerte en cas de données invalides.



03. CONCEPTION REALISATION DE LA SOLUTION

3.4 Déroulement de l'exécution (logique du programme)

- L'utilisateur est face à une interface dont on a 3 boutons , algorithme de RO ou il peut choisir l'algo à traiter, bouton Entrer ou on a des informations sur chaque algo et son utilité, et bouton quitter
- quand L'utilisateur choisit un algo , il entre les données nécessaires (graphe, tableau de coûts...).



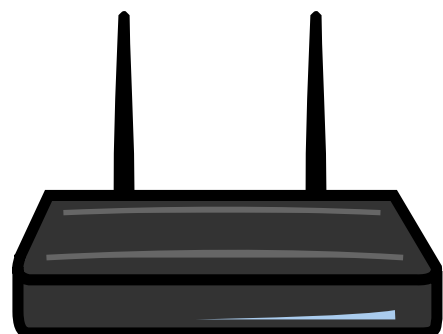
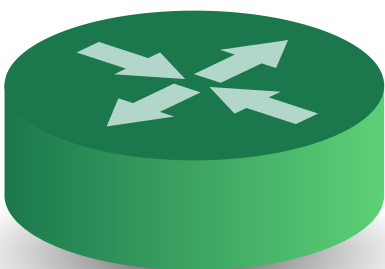
- Il clique sur un bouton pour lancer l'algorithme choisi.
- Le programme appelle la fonction correspondante dans le module algo.py.
- Une fois le traitement terminé, les résultats sont affichés dans l'interface.
- Si un problème survient (ex : matrice incorrecte), un message d'erreur est affiché.



03. CONCEPTION REALISATION DE LA SOLUTION

3.5 Résultats obtenus:

- Algorithme de Dijkstra: Visualise le plus court chemin dans un graphe pondéré, utile pour le routage IP.
- Algorithme de Bellman-Ford : Gère les graphes avec poids négatifs, robuste pour les topologies complexes sans cycles négatifs.
- Algorithme de Ford-Fulkerson : Calcule le flot maximal dans un graphe orienté, utilisé pour l'optimisation de bande passante et la gestion du trafic.
- Algorithme de Kruskal : Génère un arbre couvrant de coût minimal, utile pour la conception de réseaux physiques (ex: câblage).
- Algorithme de Welsh-Powell : Algorithme de coloration de graphes pour minimiser les interférences dans l'allocation de fréquences.
- Méthode du Potentiel-Métra : Optimise les tâches dans un graphe ordonnancé, utile pour la planification de projets complexes en télécoms/IT.
- Méthode du Coin Nord-Ouest : Fournit une solution initiale simple pour les problèmes de distribution et de routage de ressources.
- Méthode du Moindre Coût : Améliore la solution initiale pour réduire les coûts de transport, utile en logistique et routage de paquets.
- Méthode Stepping Stone : Vérifie et optimise les solutions de transport, complétant les deux méthodes précédentes.



04. RÉSULTATS

EXÉCUTION DE L'ALGORITHME DE DIJKSTRA

Jeux & Télécoms

Algorithmme de Dijkstra

Nombre de nœuds :

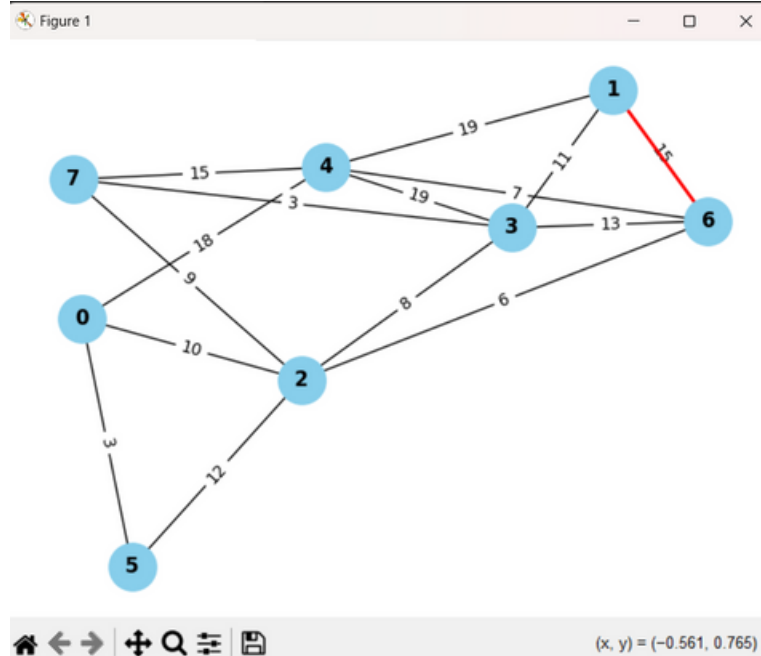
Point de départ (ex: 0) :

Point d'arrivée (ex: 4) :

Lancer Dijkstra

✕ Quitter

Optimisation du routage IP : l'algorithme de Dijkstra trouve le chemin le plus court, ce qui permet une meilleure efficacité dans la transmission des données.



Résultat

i Chemin : [6, 1]
Coût total : 15

OK

EXÉCUTION DE L'ALGORITHME DE KRUSKAL:

Kruskal Télécoms

Algorithmme de Kruskal

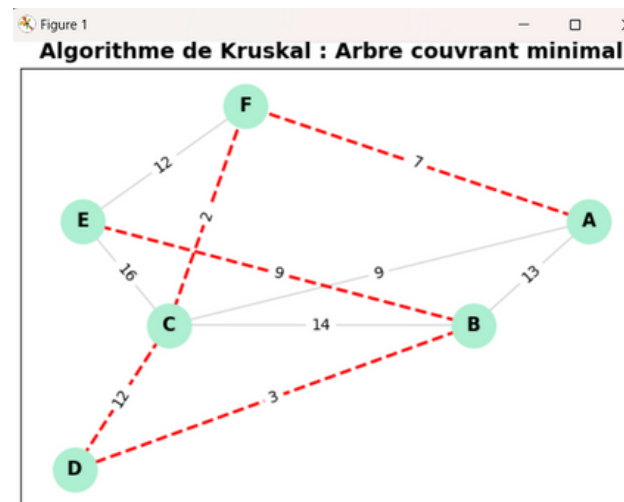
Nombre de nœuds :

Lancer l'algorithme

✕ Quitter

Dans le contexte des réseaux de télécommunications, l'algorithme de Kruskal permet de minimiser le coût d'installation de la fibre optique tout en garantissant la connexion entre les sites.

i Arbre couvrant minimal :
C - F : 2
B - D : 3
A - F : 7
B - E : 9
C - D : 12
Coût total : 33



04. RÉSULTATS

EXÉCUTION DE L'ALGORITHME DE WELSH POWEL

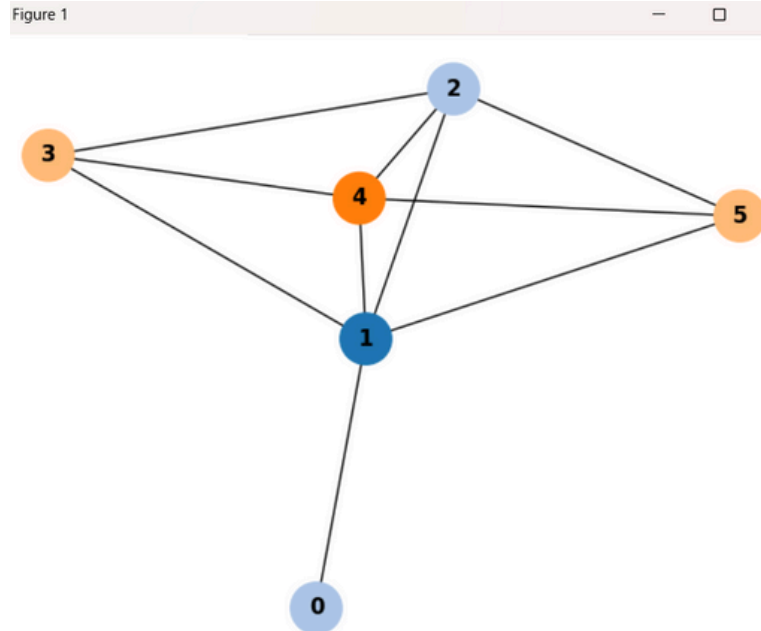
Algorithme de Welsh-Powell

Nombre de nœuds (≥ 2) :

Lancer l'algorithme

X Quitter

Contexte des réseaux de télécommunications, l'algorithme de Welsh-Powell permet d'optimiser l'utilisation des fréquences pour éviter les interférences entre stations.



EXÉCUTION DE L'ALGORITHME DE POTENTIEL METRA:

Potentiel de Métra (CPM)

Nombre de tâches (nœuds ≥ 2) :

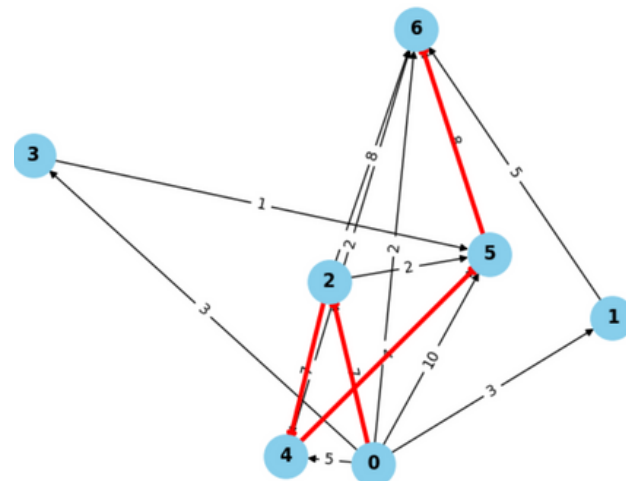
Lancer l'algorithme

X Quitter

En réseaux & télécoms, le potentiel de Métra permet de planifier les tâches critiques comme l'installation de fibre, les interventions ou les mises en service, en identifiant le chemin le plus long (durée minimale totale).

Résultat

Chemin critique : [0, 2, 4, 5, 6]
Durée minimale : 26



04. RÉSULTATS

EXÉCUTION DE L'ALGORITHME DE FORD FULKERSON

erson (Flot Max)

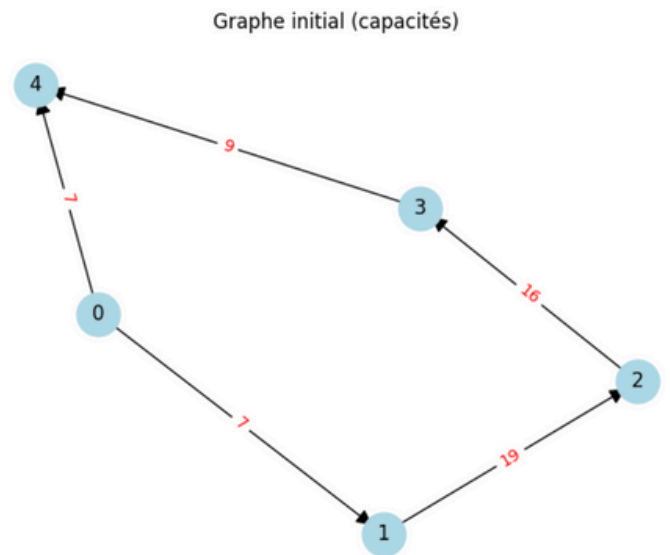
Algorithme Ford-Fulkerson

Nombre de nœuds (≥ 2) :

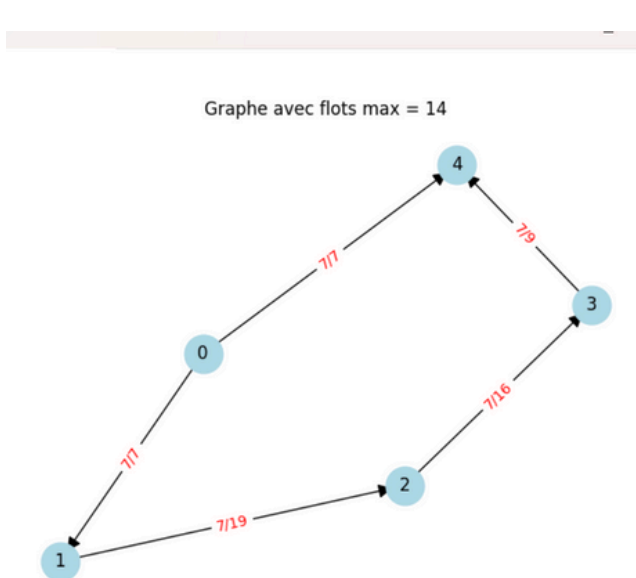
Calculer flot max

X Quitter

En télécom, Ford-Fulkerson permet d'optimiser le flux de données entre source et destination sur un réseau.



GRAPHE APRES CALCUL DE FLOT



Arcs saturés

Arcs saturés (flot = capacité max)

Arc 0 $\rightarrow$ 1 : flot = capacité = 7
Arc 0 $\rightarrow$ 4 : flot = capacité = 7

Fermer

04. RÉSULTATS

EXÉCUTION DE L'ALGORITHME DE BELLMAN FORD

Algorithme de Bellman-Ford

Nombre de sommets (≥ 2) :

Sommet de départ (source) :

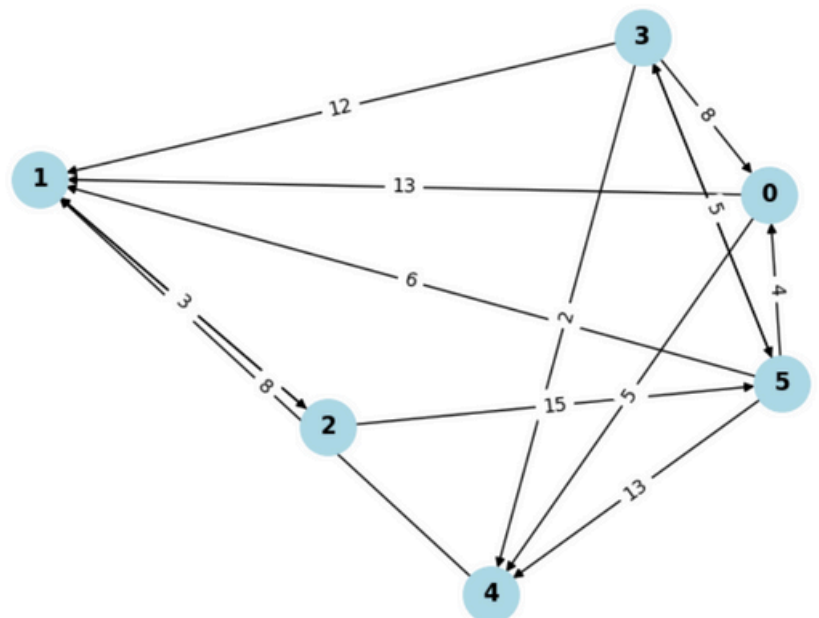
En réseaux & télécoms, Bellman-Ford est utilisé pour calculer les chemins les plus courts dans des protocoles de routage comme RIP, même avec des conditions complexes (latence, distance, surcharge). Ici, seuls des poids positifs sont utilisés.

Figure 1

Résultat

i Distances depuis le sommet 2 :

- Vers 2 : 0
- Vers 1 : 3
- Vers 5 : 15
- Vers 0 : 19
- Vers 3 : 20
- Vers 4 : 22



ON PEUT REMARQUER QUE CHAQUE ALGORITHME A UNE PETITE DESCRIPTION ET L'INTERET POUR LE RENDRE MIEUX COMPREHENSIBLE

04. RÉSULTATS

EXÉCUTION DE L'ALGORITHME DE NORDS OUEST:

fenêtre du Coin Nord-Ouest

Méthode du Coin Nord-Ouest

Nombre de sources :

Nombre de destinations :

Lancer l'algorithme

X Quitter

En télécom, cette méthode permet de démarrer une allocation de ressources simple entre centres d'émission et de réception.

Figure 1

□ Méthode Nord-Ouest — Coût total = 3239

S/D	D1	D2	D3	D4	Σ
S1	20 (47)	13 (32)	20	9	79
S2	18	18 (39)	3 (44)	10	83
S3	12	14	11 (46)	3 (27)	73
S4	18	11	18	6 (77)	77
Σ	47	71	90	104	

EXÉCUTION DE L'ALGORITHME DE MOINDRE COUT:

fenêtre du Coin Nord-Ouest

Méthode du Moindre Coût

Nombre de sources :

Nombre de destinations :

Lancer l'algorithme

X Quitter

En télécom, cet algorithme permet d'allouer des ressources comme la bande passante entre des sites tout en minimisant les coûts de transmission.

Figure 1

□ Méthode du Moindre Coût — Coût total : 3078

S/D	D1	D2	D3	D4	
S1	9 (81)	13 (5)	16	19	S=86
S2	16	13 (17)	19 (71)	16	S=88
S3	12	9 (39)	13	8 (12)	S=51
S4	17	17	9	3 (89)	S=89
	D=81	D=61	D=71	D=101	

04. RÉSULTATS

EXÉCUTION DE L'ALGORITHME DE STEPPING STONE

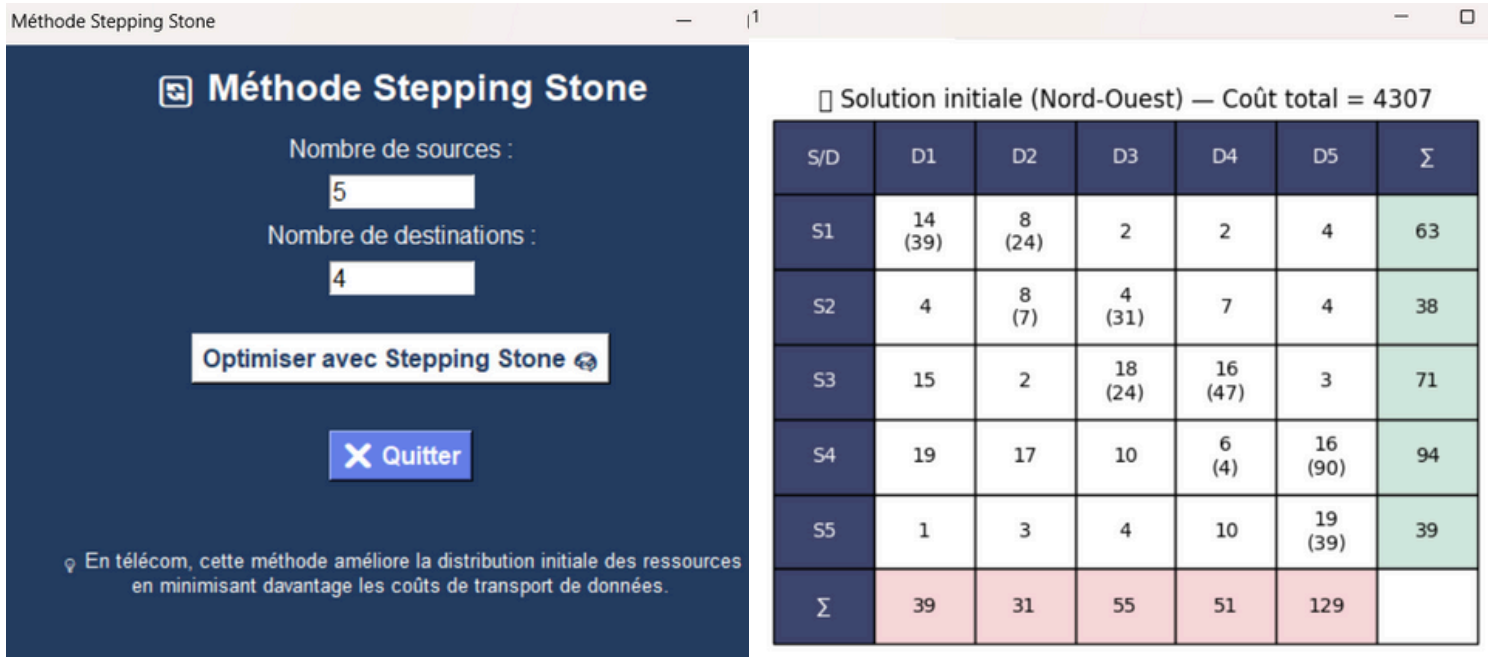


Figure 1

Solution optimisée (Stepping Stone) — Coût total = 1337

S/D	D1	D2	D3	D4	D5	Σ
S1	14	8	2 (12)	2	4 (51)	63
S2	4	8	4	7	4 (38)	38
S3	15	2 (31)	18	16	3 (40)	71
S4	19	17	10 (43)	6 (51)	16	94
S5	1 (39)	3	4	10	19	39
Σ	39	31	55	51	129	

L'ALGORITHME STEPPING STONE ESSAIE D'OPTIMISER LE COUT CALCULER APRES L'ALGO DE NORD OUEST POUR DIMINUER CE DERNIER , CE QUI EST L'INTERET

BONUS

LA PAGE ENTRER TOUTE ENTIER

Algorithmes & Réseaux

◊ Dijkstra

✦ Définition :

Trouve le plus court chemin entre un nœud source et les autres dans un graphe pondéré.

🔗 Application :

Utilisé dans le routage IP pour déterminer le chemin optimal entre deux routeurs.

◊ Kruskal

✦ Définition :

Construit un arbre couvrant minimal à partir d'un graphe pondéré.

🔗 Application :

Permet d'optimiser les connexions dans les réseaux tout en réduisant le coût total.

◊ Welsh-Powell

✦ Définition :

Coloration de graphe minimisant le nombre de couleurs sans conflit entre voisins.

🔗 Application :

Répartition efficace des fréquences sans interférence dans les réseaux cellulaires.

◊ Moindre Coût

✦ Définition :

Méthode d'allocation initiale pour résoudre les problèmes de transport à coût minimal.

🔗 Application :

Répartit efficacement les ressources (comme la bande passante ou la capacité).

◊ Nord-Ouest

✦ Définition :

Méthode de base pour créer une solution initiale d'un problème de transport.

🔗 Application :

Utile dans la gestion des ressources dans les réseaux logistiques ou télécoms.

◊ Stepping Stone

✦ Définition :

Méthode d'optimisation pour améliorer une solution initiale de transport.

🔗 Application :

Réduit les coûts de transmission dans les réseaux distribués.

◊ Ford-Fulkerson

✦ Définition :

Trouve le flux maximum dans un réseau orienté entre une source et un puits.

🔗 Application :

Optimise l'utilisation de la bande passante ou des flux de données.

◊ Potentiel de Métra

✦ Définition :

Méthode d'ordonnancement des tâches avec contraintes de précédence.

🔗 Application :

Gestion de projets réseaux avec délais et dépendances techniques.

DEFINITION ET
APPLICATION DE
CHAQUE ALGORITHME

5

Résultats, conclusion et perspectives

5.1 Résultats obtenus et comparaison avec les objectifs

Le projet a abouti à la création d'une application graphique intuitive et fonctionnelle permettant d'exécuter plusieurs algorithmes classiques de recherche opérationnelle. Les objectifs fixés en début de projet ont été atteints :

- Interface unifiée en Tkinter, avec un design sombre personnalisé ;
- Implémentation de 9 algorithmes
- Visualisation graphique via Matplotlib pour les algorithmes qui s'y prêtent ;
- Génération automatique de graphes et matrices de coûts ;
- Affichage clair des résultats (coûts, allocations, chemins, etc.).
-

ALGORITHMES



- Dijkstra : pour le chemin le plus court ;
- Kruskal : pour l'arbre couvrant minimal ;
- Welsh-Powell : pour la coloration de graphe ;
- Méthode du Moindre Coût, Nord-Ouest, et Stepping Stone : pour les problèmes de transport ;
- ford-Fulkerson : pour le flot maximum ;
- Méthode du Potentiel de Métra : pour l'ordonnancement de projets.



5

Résultats, conclusion et perspectives

5.2 Apports personnels et compétences acquises

- Approfondir mes compétences en ALGORITHMES programmation Python, notamment en utilisant des bibliothèques comme Tkinter, NetworkX, et Matplotlib.
- Appliquer concrètement des algorithmes de recherche opérationnelle à des problèmes réalistes dans le domaine des réseaux et télécommunications.
- Concevoir une interface graphique ergonomique avec des messages clairs, une esthétique cohérente, et une navigation intuitive.
- Structurer un projet logiciel en respectant une méthodologie rigoureuse.

5.3 Limites du projet

Malgré les résultats obtenus, certaines limites subsistent :

- ✗ L'application ne permet pas encore l'import de données réelles (topologies réseau, matrices externes) ;
- ✗ Les algorithmes ne sont pas encore modulaires ni testés avec des cas très complexes ou de grande échelle ;
- ✗ Il n'y a pas de sauvegarde ou d'historique des résultats dans l'interface actuelle.

5

Résultats, conclusion et perspectives

5.4 Améliorations possibles et perspectives futures

GESTION DES DONNEES

Ajout d'un système d'import/export pour les graphes et matrices (via fichiers CSV, JSON, etc.) ;

ARCHITECTURE LOGIQUE

Modularisation du code pour une réutilisabilité optimale et une extension facile à d'autres algorithmes ;

PERSISTANCE DES DONNEES

Ajout d'une base de données pour enregistrer les sessions, les résultats, ou les préférences utilisateur ;

OPTIMISATION AVANCEE

Intégration de moteurs d'optimisation plus avancés (ex : PuLP, OR-Tools) pour aller plus loin dans la recherche opérationnelle.

Conclusion



Ce projet a permis de développer une application Python regroupant plusieurs algorithmes fondamentaux de la théorie des graphes et de l'optimisation. Bien que le développement se soit limité à l'implémentation pure via Python et ses bibliothèques, sans passer par une modélisation formelle (MCD, UML), les résultats obtenus sont probants. Les algorithmes étudiés trouvent des applications concrètes dans le domaine des réseaux et télécommunications, notamment dans l'optimisation des chemins, la gestion des ressources, ou encore la planification des projets. Ce travail m'a permis de renforcer mes compétences en programmation algorithmique, en manipulation de structures de données, et surtout de mieux comprendre l'utilité de ces outils dans des contextes réels.

En perspective, plusieurs pistes d'amélioration sont envisageables : ajouter une interface utilisateur plus interactive, intégrer des visualisations graphiques dynamiques ou encore connecter l'application à de vraies données réseau pour des simulations plus concrètes.

RESSOURCES PYTHON ET BIBLIOTHÈQUES

- Python Software Foundation. Python Language Reference, version 3. <https://www.python.org/doc/>
- NetworkX Developers. NetworkX Documentation. <https://networkx.org/documentation/>
- Matplotlib Developers. Matplotlib: Visualization with Python. <https://matplotlib.org/>
- GeeksforGeeks. Data Structures and Algorithms in Python. <https://www.geeksforgeeks.org/>
- Real Python. Python Tutorials and Code Examples. <https://realpython.com/>
-

ALGORITHMES ET THÉORIE DES GRAPHES

- **Implementing the Dijkstra Algorithm in Python :**
<https://www.datacamp.com/tutorial/dijkstra-algorithm-in-python>
- <https://www.geeksforgeeks.org/kruskals-algorithm-in-python/>
- https://github.com/FarahBaraket-03/Graphe_Algo_WelshPowell/blob/main/test.py
- <https://www.imo.universite-paris-saclay.fr/~ruette/mathsdiscretes/polygraph-Sigward.pdf>